

HW4 отчёт

ФИО: Пономарев Николай Юрьевич **Группа:** 234 **Email:** niiuponomarev@edu.hse.ru

Цель работы

Разработать многопользовательское клиент-серверное файловое хранилище, работающее по протоколу TCP. Сервер принимает неограниченное число клиентов, клиент синхронизирует заданную локальную директорию с серверной, передавая только отсутствующие или изменённые файлы, используя несколько параллельных TCP-соединений. На практике освоить шаблон length-prefix framing поверх TCP, механизм ByteToMessageDecoder в Netty и понаблюдать поведение TCP (MSS, окно, slow start, congestion) на реальной передаче.

Претендуемая оценка

Я претендую на оценку до 8 баллов (Java / Netty: базовая функциональность 6 баллов + параллельная отправка нескольких файлов +2 балла). Не был использован генеративный ИИ.

Выбранные язык и фреймворки

- Язык: Java 21
- Фреймворк Netty 4.1.115.Final (NIO, [ByteToMessageDecoder](#) / [MessageToByteEncoder](#)).
- Также использовал Jackson 2.18.2 для JSON-сериализация, SLF4J для логирования.

Архитектура решения

Сервер принимает соединения через ServerInitializer, который вешает на каждый канал пайплайн FrameDecoder -> FrameEncoder -> ServerSession. Файлы складываются в `<storageDir>/<clientId>/`.

Клиент поддерживает команды sync или quit. По команде sync:

1. Открывает одно контрольное TCP-соединение, шлёт `HELLO + FILE_LIST`, получает `SYNC_PLAN`.
2. Открывает N data-соединений. Каждый воркер держит ровно одно TCP-соединение, тянет имена файлов из общей очереди и шлёт `HELLO -> FILE_BEGIN -> FILE_CHUNK -> FILE_END`, дожидаясь `FILE_ACK` перед следующим файлом. Когда очередь пуста — соединение закрывается.

Также реализован параллелизм и одновременно вверх летят несколько файлов в независимых TCP-потоках.

Ключевые фрагменты кода

Декодер кадров — [FrameDecoder.java](#)

```
public class FrameDecoder extends ByteToMessageDecoder {
    private static final int LENGTH_FIELD = 4;
    private static final int MAX_FRAME_LENGTH = 1024 * 1024;

    @Override
```

```

protected void decode(ChannelHandlerContext ctx, ByteBuf in, List<Object> out)
{
    if (in.readableBytes() < LENGTH_FIELD) return;
    in.markReaderIndex();
    int length = in.readInt();
    if (length < 1 || length > MAX_FRAME_LENGTH) {
        ctx.close();
        throw new IllegalStateException("invalid frame length: " + length);
    }
    if (in.readableBytes() < length) {
        in.resetReaderIndex();
        return;
    }
    int typeCode = in.readByte() & 0xFF;
    byte[] payload = new byte[length - 1];
    in.readBytes(payload);
    out.add(new Frame(MessageType.fromCode(typeCode), payload));
}
}

```

Полностью реализует шаблон со слайда из лекции: ждём заголовок длины, если данных не хватает — откатываемся к точке `markReaderIndex`, не теряя прочитанное; иначе достаём ровно `length` байт и выдаём готовое сообщение. Никакой высокоуровневой абстракции, в TCP явно уходит `byte[]` через `FrameEncoder`.

Серверная сессия — `ServerSession`

Обрабатывает входящие фреймы (state machine). При `FILE_BEGIN` открывает файл `<storageDir>/<clientId>/<name>.part`, потоково пишет `FILE_CHUNK` через `OutputStream`, по `FILE_END` сверяет полученный размер и CRC32, атомарно переименовывает `.part` -> финальное имя и отправляет `FILE_ACK`. При обрыве соединения временный `.part` удаляется (`closeCurrentSilently`).

Оркестратор синхронизации — `SyncOrchestrator`

```

public void syncOnce() throws Exception {
    DirectoryScanner scanner = new DirectoryScanner(config.watchDir);
    List<FileMeta> files = scanner.scan();
    List<String> plan = exchangePlan(files);
    if (plan.isEmpty()) return;

    BlockingQueue<String> queue = new LinkedBlockingQueue<>(plan);
    int workers = Math.min(config.maxConnections, plan.size());
    ExecutorService pool = Executors.newFixedThreadPool(workers);
    Future<?>[] futures = new Future<?>[workers];
    for (int i = 0; i < workers; i++) {
        futures[i] = pool.submit(new UploadWorker(i, config, scanner, queue,
group));
    }
    for (Future<?> f : futures) f.get();
}

```

```
pool.shutdown();  
}
```

`exchangePlan` открывает контрольное соединение, шлёт `HELLO` + `FILE_LIST` и через `CompletableFuture` ждёт `SYNC_PLAN`. После этого имена попадают в общую очередь, и поднимается ровно столько воркеров, сколько нужно для плана (но не больше `maxConnections`).

Воркер — `UploadWorker`

```
String name;  
while ((name = queue.poll()) != null) {  
    Path file = scanner.resolve(name);  
    long size = Files.size(file);  
    long crc = Checksum.crc32(file);  
    CompletableFuture<FileAckPayload> ackFuture = new CompletableFuture<>();  
    collector.expect(name, ackFuture);  
  
    ch.writeAndFlush(new Frame(FILE_BEGIN, Codec.toJson(new FileBeginPayload(name,  
size, crc)))).sync();  
    try (InputStream in = Files.newInputStream(file)) {  
        byte[] buf = new byte[CHUNK_SIZE]; // 64 KiB  
        int n;  
        while ((n = in.read(buf)) != -1) {  
            byte[] chunk = (n == buf.length) ? buf.clone() : Arrays.copyOf(buf,  
n);  
  
            while (!ch.isWritable()) Thread.sleep(1); // back-pressure  
            ch.writeAndFlush(new Frame(FILE_CHUNK, chunk));  
        }  
    }  
    ch.writeAndFlush(Frame.empty(FILE_END)).sync();  
    ackFuture.get(ACK_TIMEOUT_SEC, TimeUnit.SECONDS);  
}
```

Воркер использует back-pressure через `ch.isWritable()`: пока Netty не успевает выталкивать буферы в OS, мы ставим тред на короткую паузу. Это исключает OOM на файлах в 1 ГБ и адекватно реагирует на медленную сеть.

Сценарий запуска

Запуск сервера

В корне проекта:

```
mvn -DskipTests -q package  
java -jar target/server.jar server-config.properties
```

Конфиг `server-config.properties`:

```
port=9090
storageDir=server-storage
```

Запуск клиента

```
java -jar target/client.jar client-config.properties
```

Конфиг `client-config.properties`:

```
clientId=                Пустой, т.к. UUID будет сгенерирован и записан обратно
watchDir=client-data
serverHost=127.0.0.1
serverPort=9090
maxConnections=4
```

В консоли клиента ввести команду `sync` для запуска синхронизации. Для выхода — `quit`.

Подготовка тестовых данных

10 файлов по 100 МБ со случайными байтами, нужно выполнить скрипт в PowerShell:

```
.\scripts\generate-test-files.ps1 -Count 10 -SizeMB 100
```

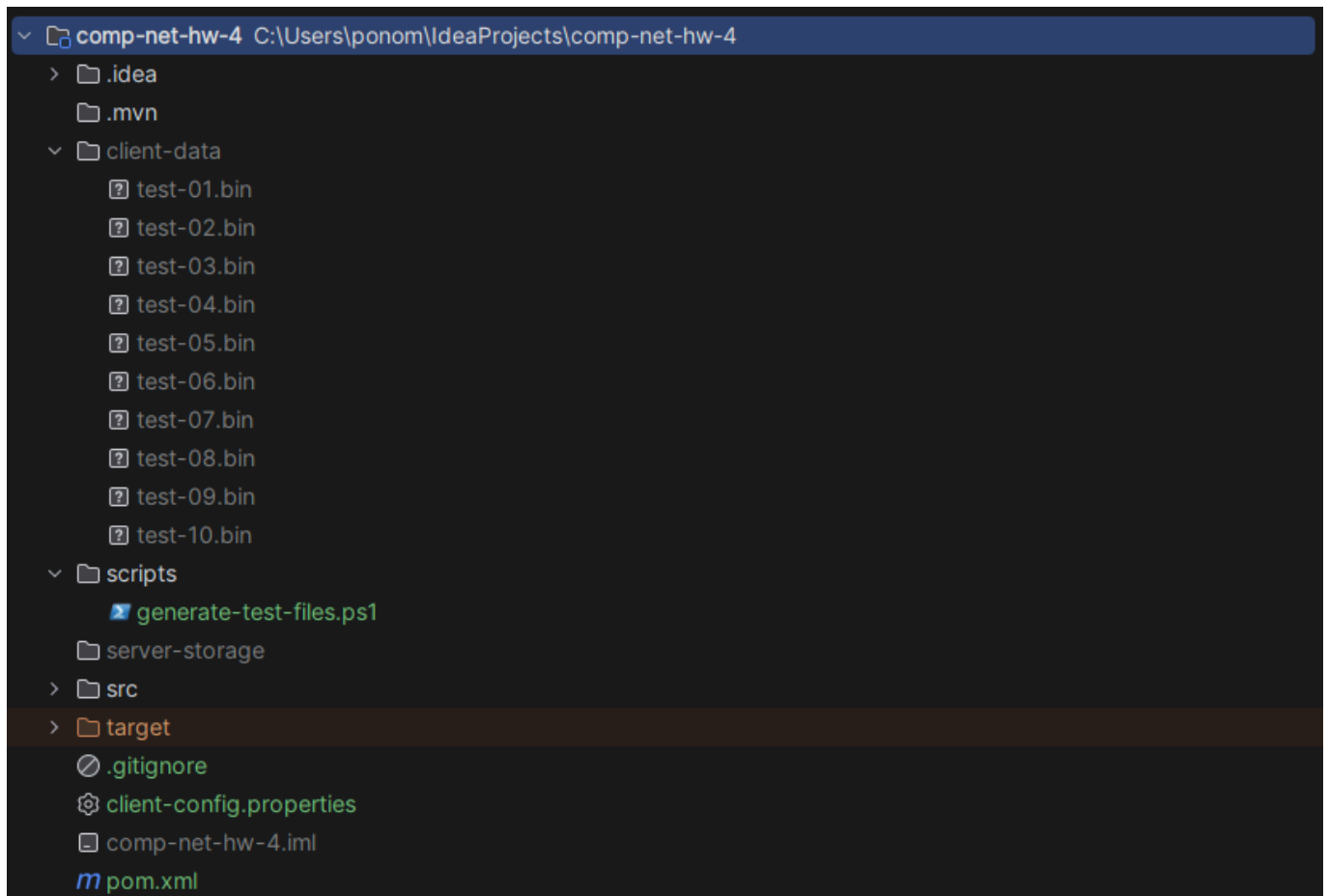
Эксперимент 1. Базовый сценарий — синхронизация 10 файлов по 100 МБ

Подготовка

1. Удалить (или переместить) папки `client-data\` и `server-storage\`.
2. Запустить генератор: `.\scripts\generate-test-files.ps1 -Count 10 -SizeMB 100`.
3. Запустить сервер.
4. Запустить клиент, ввести `sync`.

Скриншоты

Скриншот 01. Состояние директорий ДО `sync`. Сделать командой `Get-ChildItem client-data, server-storage -Recurse | Format-Table FullName, Length` в PowerShell. Видно, что у клиента 10 файлов по ~100 МБ, у сервера — пусто.



Скриншот 02. Лог клиента после **sync**.

```
C:\WINDOWS\system32\cmd. x + v
WARNING: sun.misc.Unsafe::allocateMemory has been called by io.netty.util.internal.PlatformDependent0$2 (file:/C:/Users/ponom/IdeaProjects/comp-net-hw-4/target/client.jar)
WARNING: Please consider reporting this to the maintainers of class io.netty.util.internal.PlatformDependent0$2
WARNING: sun.misc.Unsafe::allocateMemory will be removed in a future release
commands: sync | quit
> sync
[main] INFO ru.hse.mydrive.client.SyncOrchestrator - scanned 10 files in C:\Users\ponom\IdeaProjects\comp-net-hw-4\client-data
[main] INFO ru.hse.mydrive.client.SyncOrchestrator - server plan: upload 10 of 10 files
[pool-1-thread-1] INFO ru.hse.mydrive.client.UploadWorker - worker 0 uploaded test-02.bin (104857600 bytes) in 1185 ms (84,39 MB/s)
[pool-1-thread-3] INFO ru.hse.mydrive.client.UploadWorker - worker 2 uploaded test-01.bin (104857600 bytes) in 1205 ms (82,99 MB/s)
[pool-1-thread-2] INFO ru.hse.mydrive.client.UploadWorker - worker 1 uploaded test-04.bin (104857600 bytes) in 1230 ms (81,30 MB/s)
[pool-1-thread-4] INFO ru.hse.mydrive.client.UploadWorker - worker 3 uploaded test-03.bin (104857600 bytes) in 1291 ms (77,46 MB/s)
[pool-1-thread-2] INFO ru.hse.mydrive.client.UploadWorker - worker 1 uploaded test-07.bin (104857600 bytes) in 1061 ms (94,25 MB/s)
[pool-1-thread-3] INFO ru.hse.mydrive.client.UploadWorker - worker 2 uploaded test-06.bin (104857600 bytes) in 1212 ms (82,51 MB/s)
[pool-1-thread-1] INFO ru.hse.mydrive.client.UploadWorker - worker 0 uploaded test-05.bin (104857600 bytes) in 1209 ms (77,58 MB/s)
[pool-1-thread-4] INFO ru.hse.mydrive.client.UploadWorker - worker 3 uploaded test-08.bin (104857600 bytes) in 1213 ms (82,44 MB/s)
[pool-1-thread-2] INFO ru.hse.mydrive.client.UploadWorker - worker 1 uploaded test-09.bin (104857600 bytes) in 836 ms (119,62 MB/s)
[pool-1-thread-3] INFO ru.hse.mydrive.client.UploadWorker - worker 2 uploaded test-10.bin (104857600 bytes) in 894 ms (111,86 MB/s)
[main] INFO ru.hse.mydrive.client.SyncOrchestrator - sync round done in 3421 ms using 4 connections
> |
```

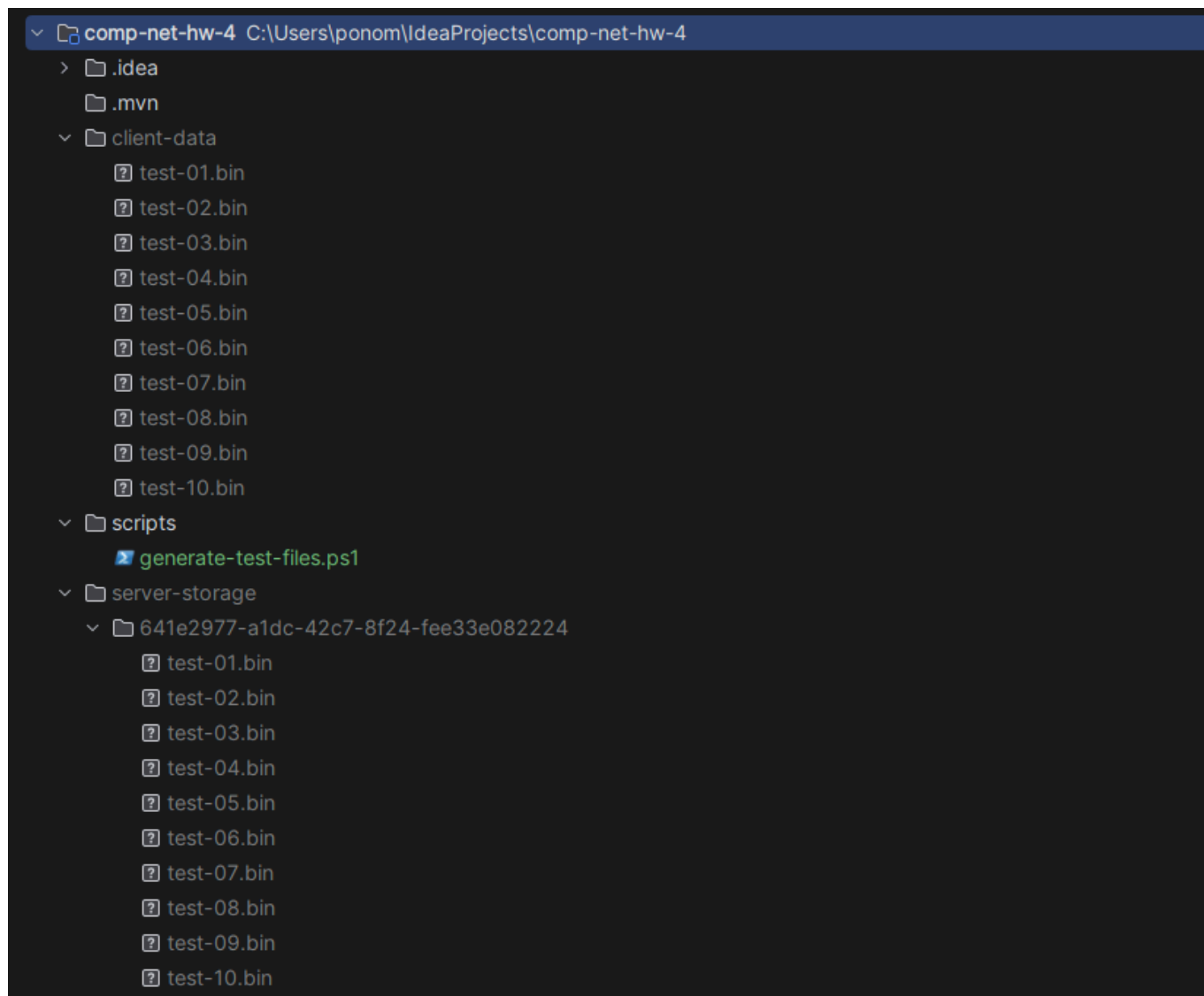
Скриншот 03. Лог сервера в этот же момент. Видны N подряд идущих сообщений **connection opened**, **HELLO**, **FILE_LIST**, **FILE_BEGIN**, **FILE_END** ok.

```

PS C:\Users\ponom\IdeaProjects\comp-net-hw-4> mvn -DskipTests -q package
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe:staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Users/ponom/apache-maven-3.9.11/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe:staticFieldBase will be removed in a future release
PS C:\Users\ponom\IdeaProjects\comp-net-hw-4> java -jar target/server.jar server-config.properties
[main] INFO ru.hse.mydrive.server.ServerMain - storage dir: C:\Users\ponom\IdeaProjects\comp-net-hw-4\server-storage
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe:allocateMemory has been called by io.netty.util.internal.PlatformDependent0$2 (file:/C:/Users/ponom/IdeaProjects/comp-net-hw-4/target/server.jar)
WARNING: Please consider reporting this to the maintainers of class io.netty.util.internal.PlatformDependent0$2
WARNING: sun.misc.Unsafe:allocateMemory will be removed in a future release
[main] INFO ru.hse.mydrive.server.ServerMain - listening on 9090
[nioEventLoopGroup-3-1] INFO ru.hse.mydrive.server.ServerSession - connection opened /127.0.0.1:58747
[nioEventLoopGroup-3-1] INFO ru.hse.mydrive.server.ServerSession - HELLO clientId=641e2977-a1dc-42c7-8f24-fee33e082224
[nioEventLoopGroup-3-1] INFO ru.hse.mydrive.server.ServerSession - FILE_LIST clientId=641e2977-a1dc-42c7-8f24-fee33e082224 files=10 toUpload=10
[nioEventLoopGroup-3-1] INFO ru.hse.mydrive.server.ServerSession - connection closed /127.0.0.1:58747
[nioEventLoopGroup-3-1] INFO ru.hse.mydrive.server.ServerSession - connection opened /127.0.0.1:58749
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - connection opened /127.0.0.1:58748
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - connection opened /127.0.0.1:58750
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - connection opened /127.0.0.1:58751
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - HELLO clientId=641e2977-a1dc-42c7-8f24-fee33e082224
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - HELLO clientId=641e2977-a1dc-42c7-8f24-fee33e082224
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - HELLO clientId=641e2977-a1dc-42c7-8f24-fee33e082224
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - HELLO clientId=641e2977-a1dc-42c7-8f24-fee33e082224
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-03.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-02.bin size=104857600
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-04.bin size=104857600
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-01.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-02.bin size=104857600
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-01.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-05.bin size=104857600
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-04.bin size=104857600
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-06.bin size=104857600
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-07.bin size=104857600
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-03.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-08.bin size=104857600
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-07.bin size=104857600
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-09.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-06.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_BEGIN clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-10.bin size=104857600
[nioEventLoopGroup-3-3] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-05.bin size=104857600
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - connection closed /127.0.0.1:58749
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-08.bin size=104857600
[nioEventLoopGroup-3-5] INFO ru.hse.mydrive.server.ServerSession - connection closed /127.0.0.1:58751
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-09.bin size=104857600
[nioEventLoopGroup-3-4] INFO ru.hse.mydrive.server.ServerSession - connection closed /127.0.0.1:58750
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - FILE_END ok clientId=641e2977-a1dc-42c7-8f24-fee33e082224 name=test-10.bin size=104857600
[nioEventLoopGroup-3-2] INFO ru.hse.mydrive.server.ServerSession - connection closed /127.0.0.1:58748

```

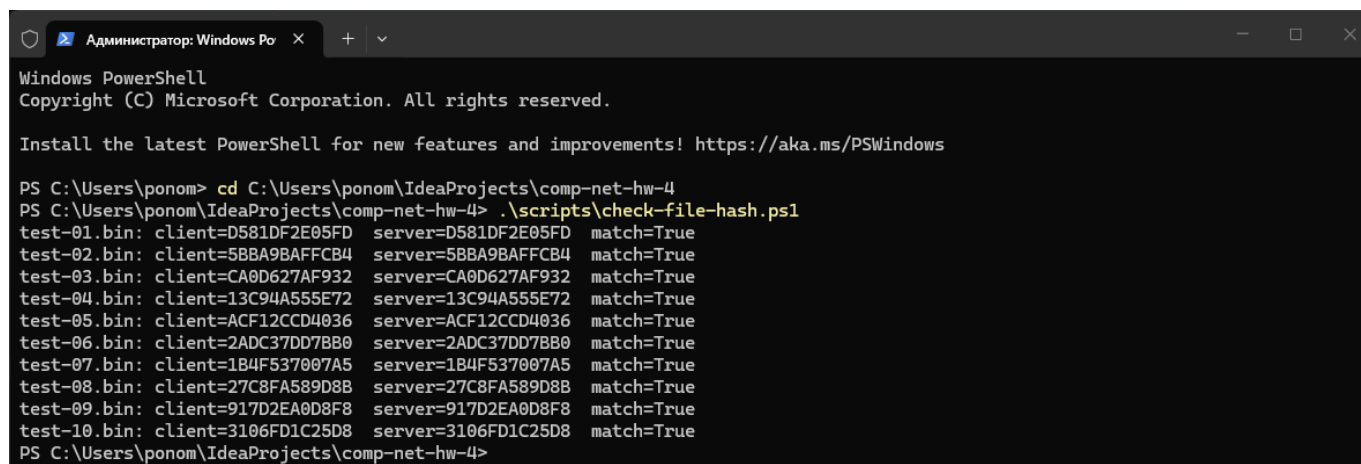
Скриншот 04. Состояние директорий после синхронизации. Та же команда, что в (01); у сервера теперь видна папка с **clientId**, в ней те же 10 файлов с теми же размерами.



Скриншот 05. Проверка побайтового совпадения через `Get-FileHash`, можно выполнить в отдельном терминале в корне проекта:

```
.\scripts\check-file-hash.ps1
```

Все хэши совпадают.



Эксперимент 2. Параллельная отправка нескольких файлов

Подготовка

- 1. В `client-config.properties` выставить `maxConnections=8`.
- 2. Опять очистить `server-storage\`.
- 3. Запустить WireShark на интерфейсе loopback (Npcap loopback adapter, фильтр `tcp.port == 9090`).
- 4. Стартовать сервер, в клиенте — `sync`.

Что именно показывает параллелизм

Видно несколько одновременных ESTABLISHED-соединений к серверному порту 9090 + 1 контрольное, но оно не успело открыться, поэтому на скриншоте их вообще всего 4. Это и есть так называемая параллельная отправка нескольких файлов.

```
PS C:\Users\ponom\IdeaProjects\comp-net-hw-4> Get-NetTCPConnection -LocalPort 9090 -State Established | Format-Table -AutoSize
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	AppliedSetting	OwningProcess
127.0.0.1	9090	127.0.0.1	57743	Established	Internet	23204
127.0.0.1	9090	127.0.0.1	57744	Established	Internet	23204
127.0.0.1	9090	127.0.0.1	57742	Established	Internet	23204
127.0.0.1	9090	127.0.0.1	57745	Established	Internet	23204

for loopback traffic capture

Ethernet		IPv4 · 1		IPv6		TCP · 10	
Адрес А	Порт А	Адрес В	Порт В	Пакеты	Байт	ИД потока	Всего пакетов
127.0.0.1	57741	127.0.0.1	9090	13	1 кБ	54	13
127.0.0.1	57742	127.0.0.1	9090	10 348	233 МБ	55	10 348
127.0.0.1	57743	127.0.0.1	9090	6 710	156 МБ	56	6 710
127.0.0.1	57744	127.0.0.1	9090	6 671	154 МБ	57	6 671
127.0.0.1	57745	127.0.0.1	9090	10 282	232 МБ	58	10 282
127.0.0.1	60966	127.0.0.1	9090	13	1 кБ	32	13
127.0.0.1	60967	127.0.0.1	9090	6 836	158 МБ	33	6 836
127.0.0.1	60968	127.0.0.1	9090	10 618	235 МБ	34	10 618
127.0.0.1	60969	127.0.0.1	9090	6 800	157 МБ	35	6 800
127.0.0.1	60970	127.0.0.1	9090	10 660	238 МБ	36	10 660

Эксперимент 3. Поведение протокола TCP

tcp.flags.syn == 1 && tcp.flags.ack == 0 && tcp.dstport == 9090

No.	Time	Source	Destination	Protocol	Length	Info
1077	64.949270700	127.0.0.1	127.0.0.1	TCP	56	60966 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
1134	65.286966700	127.0.0.1	127.0.0.1	TCP	56	60967 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
1137	65.287288600	127.0.0.1	127.0.0.1	TCP	56	60968 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
1140	65.287482600	127.0.0.1	127.0.0.1	TCP	56	60969 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
1143	65.287949300	127.0.0.1	127.0.0.1	TCP	56	60970 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
38381	145.502341900	127.0.0.1	127.0.0.1	TCP	56	57741 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
38394	145.700431400	127.0.0.1	127.0.0.1	TCP	56	57742 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
38397	145.700739700	127.0.0.1	127.0.0.1	TCP	56	57743 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
38400	145.701031800	127.0.0.1	127.0.0.1	TCP	56	57744 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
38403	145.701748700	127.0.0.1	127.0.0.1	TCP	56	57745 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2

[Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]

Null/Loopback
Family: IP (2)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differerentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 52
Identification: 0x5f77 (24439)
010. = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: TCP (6)
Header Checksum: 0x9d4a [validation disabled]
[Header checksum status: Unverified]
Source Address: 127.0.0.1
Destination Address: 127.0.0.1
[Stream index: 0]
Transmission Control Protocol, Src Port: 57744, Dst Port: 9090, Seq: 0, Len: 0

tcp.analysis.flags && tcp.port == 9090

No.	Time	Source	Destination	Protocol	Length	Info
67757	149.594994400	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57742 [ACK] Seq=103
67763	149.596852200	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57745 → 9090 [ACK]
67765	149.596870500	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57742 → 9090 [ACK]
67767	149.596889000	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57745 [ACK] Seq=103
67768	149.596906800	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57742 [ACK] Seq=103
67779	149.598897300	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57745 → 9090 [ACK]
67781	149.598923200	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57742 → 9090 [ACK]
67783	149.598938500	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57745 [ACK] Seq=103
67784	149.598959500	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57742 [ACK] Seq=103
67786	149.599193000	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57742 → 9090 [ACK]
67788	149.599214900	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57742 [ACK] Seq=103
67791	149.601006400	127.0.0.1	127.0.0.1	TCP	65539	[TCP Previous segment not captured] 57745 → 9090 [ACK]
67794	149.601062700	127.0.0.1	127.0.0.1	TCP	44	[TCP ACKed unseen segment] 9090 → 57745 [ACK] Seq=103
67806	149.601378100	127.0.0.1	127.0.0.1	TCP	44	[TCP Window Update] 9090 → 57745 [ACK] Seq=103 Ack=25
67808	149.601398100	127.0.0.1	127.0.0.1	TCP	44	[TCP Window Update] 9090 → 57745 [ACK] Seq=103 Ack=25
67812	149.601453500	127.0.0.1	127.0.0.1	TCP	44	[TCP Window Update] 9090 → 57745 [ACK] Seq=103 Ack=25
67847	149.605066100	127.0.0.1	127.0.0.1	TCP	44	[TCP Window Update] 9090 → 57742 [ACK] Seq=103 Ack=25
67850	149.605088500	127.0.0.1	127.0.0.1	TCP	44	[TCP Window Update] 9090 → 57742 [ACK] Seq=103 Ack=25

Frame 67808: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_{Loopback}

Section number: 1
Interface id: 0 (\Device\NPF_{Loopback})
Encapsulation type: NULL/Loopback (15)
Arrival Time: May 11, 2026 13:26:49.061475300 RTZ 2 (зима)
UTC Arrival Time: May 11, 2026 10:26:49.061475300 UTC
Epoch Arrival Time: 1778495209.061475300
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 11.200 microseconds]
[Time delta from previous displayed frame: 20.000 microseconds]
[Time since reference or first frame: 2 minutes, 29.601398100 seconds]
Frame Number: 67808
Frame Length: 44 bytes (352 bits)
Capture Length: 44 bytes (352 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: null:ip:tcp]
Character encoding: ASCII (0)
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]

wireshark_Adapter for loopback traffic capture801603.pcapng

Пакеты: 80094 · Отображено: 10811 (13.5%)

Профиль: Default

- MSS и Window. В пакетах `[SYN]` и `[SYN, ACK]` обе стороны анонсируют свой MSS в TCP Options; итоговый MSS соединения — минимальный из двух. На loopback'e (`tcp.dstport == 9090`, фильтр `tcp.flags.syn == 1`) MSS обычно очень крупный (~65495 байт), потому что loopback-интерфейс не имеет MTU ethernet'a. Там же видна опция Window Scale (множитель окна, чтобы преодолеть лимит 64 КБ из 16-битного поля). По мере приёма данных Advertised Window увеличивается — получатель успевает забирать данные из сокетного буфера быстрее, чем они приходят.
- Также были выявлены аномалии TCP (retransmission, duplicate ACK, out-of-order). На loopback'e таких событий, как правило, нет — пакеты не маршрутизируются и не борются за очередь, congestion не возникает.
- При `maxConnections > 1` суммарная пропускная способность синхронизации заметно выше: у каждого TCP-соединения свой cwnd и своя фаза slow start, поэтому несколько параллельных потоков быстрее «прогревают» канал, чем один.